

JC4004 – Computational Intelligence

Sessions 23 & 24: Domain adaptation

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)

Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)

Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

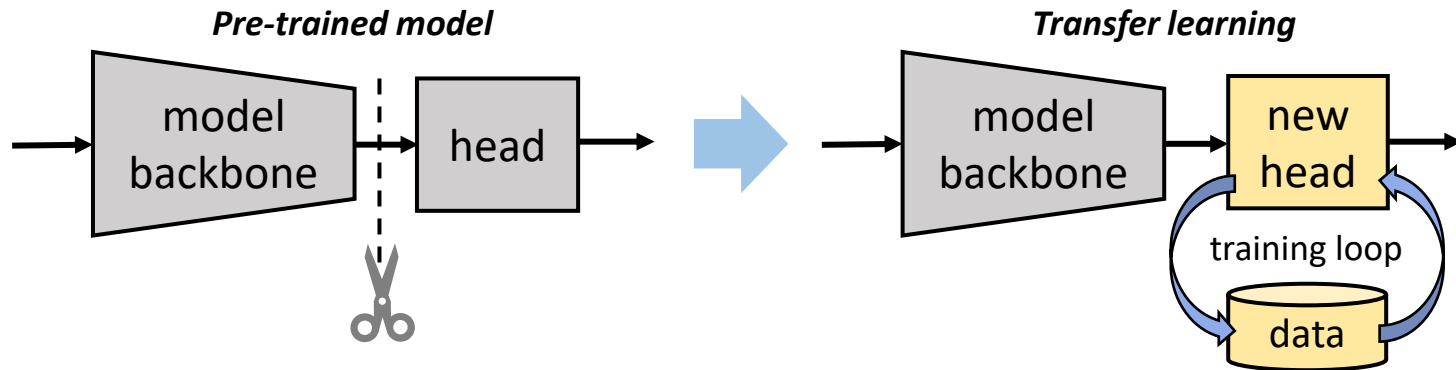
Oct-Dec 2025

Motivation for domain adaptation

- Deep neural networks make accurate predictions only for samples that are sufficiently similar to the samples used for training
 - For example, a model trained for recognising photos of birds cannot be used for recognising drawings of fish: photos and drawings represent different *domains*
- Models trained in one domain can often be adapted to other domains
 - *Transfer learning*: re-train the prediction head on different domain data using the features from a pre-trained model
 - *Domain adaptation*: adjust a pre-trained model so that the distribution of features is shifted towards the requirements of the target domain

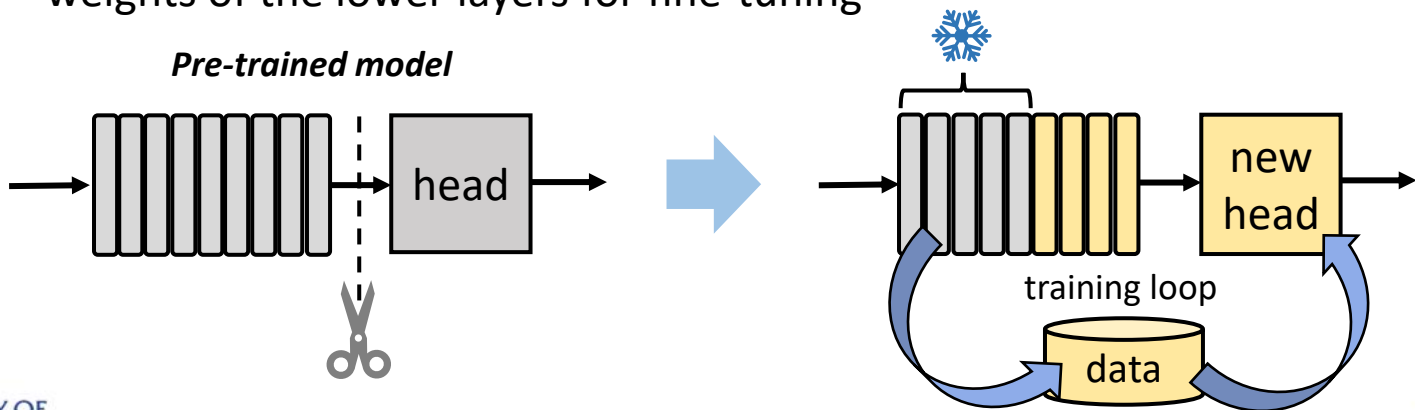
Transfer learning

- *Transfer learning*: features extracted with a pre-trained model backbone
 - The prediction head is replaced and trained to perform the new task
 - Can be trained with a smaller dataset than the full model from scratch



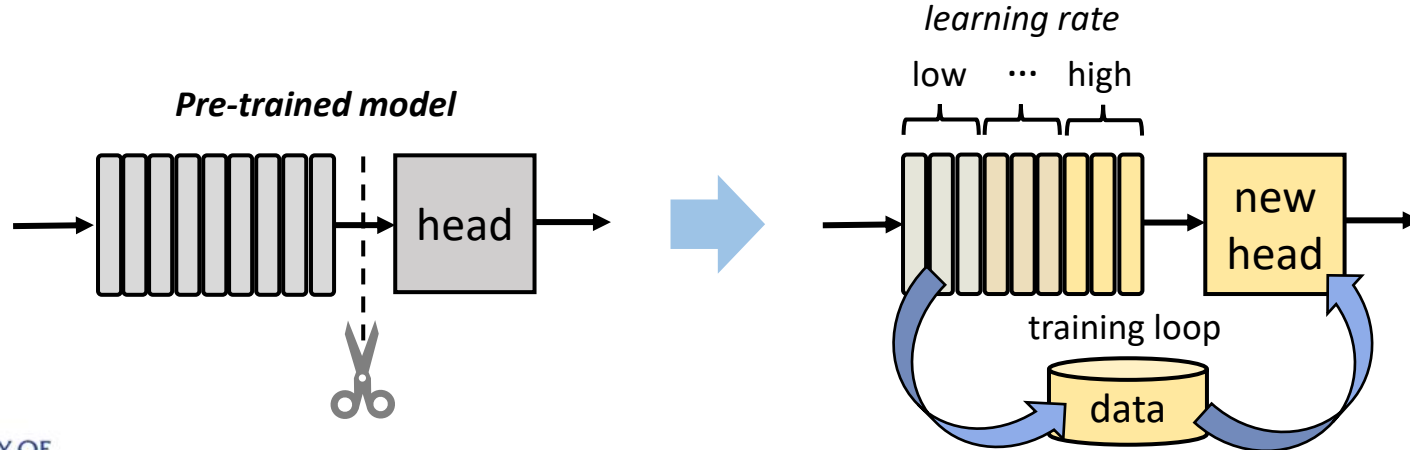
Fine-tuning with partially frozen weights

- Often, transfer learning results can be improved by *fine-tuning* also the weights of the model backbone
 - Typically, the first layers represent generic features whereas the last layers represent more task-specific features: a common strategy is to freeze the weights of the lower layers for fine-tuning



Fine-tuning with differential learning rate

- Instead of freezing the lowest layers, it can be beneficial to set the learning rate for the layers individually
 - Bottom layers fine-tuned using lower learning rate than the top layers



Pre-trained models

- Several different pre-trained deep models are publicly available and can be used for transfer learning or fine-tuning
 - For example, general image classification models trained on the large *ImageNet* dataset can be repurposed for more specific classification tasks
 - It is essential to choose a model that has been trained on a dataset sufficiently similar to the target domain
- Pre-trained models can be searched in *model zoos*, for example:
 - TensorFlow Model Garden: https://www.tensorflow.org/guide/model_garden
 - PyTorch TorchVision: <https://pypi.org/project/torchvision/>
 - Hugging Face: <https://huggingface.co/>

Training from scratch vs. transfer learning

- The optimal training strategy depends always on the model and data
 - Some common differences between training from scratch and transfer learning summarised below (but note that there can be exceptions!)

Training from scratch	Transfer learning / fine-tuning
Requires a lot of training data	Feasible with small datasets
<i>Converges slowly</i> : often hundreds of epochs required	<i>Converges fast</i> : a few epochs can be sufficient for convergence
Start with high learning rate	Start with low learning rate

Transfer learning example

- Many deep learning software libraries support transfer learning
- In the following we show a transfer learning example using Tensorflow and Keras, tested in Jupyter notebook
- First, install Tensorflow and the datasets:

```
!pip install tensorflow-gpu==2.11.0  
!pip install -U tensorflow_datasets
```


Transfer learning example

- Import libraries and load the used dataset

```
1 import tensorflow as tf
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 import random
7 import tensorflow_datasets as tfds
8
9 (train_examples, validation_examples), info = tfds.load(
10     'cats_vs_dogs',
11     split = ('train[:80%]', 'train[80%:]'),
12     with_info = True,
13     as_supervised = True
14 )
```

Transfer learning example

- Import libraries and load the used dataset

```
1 import tensorflow as tf
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 import random
7 import tensorflow_datasets as tfds
8
9 (train_examples, validation_examples), info = tfds.load(
10     'cats_vs_dogs',
11     split = ('train[:80%]', 'train[80%:]'),
12     with_info = True,
13     as_supervised = True
14 )
```

Load “cats_vs_dogs” image dataset from tensorflow dataset repository

Transfer learning example

- Define image preprocessing and obtain training and validation batches from the dataset

```
15 def format_image(image, label):
16     image = tf.image.resize(image, (224, 224))/255.0
17     return image, label
18
19 num_examples = info.splits['train'].num_examples
20
21 train_batches = train_examples.cache().shuffle(
22     num_examples//4).map(format_image).batch(32).prefetch(1)
23 validation_batches = validation_examples.map(
24     format_image).batch(32).prefetch(1)
25
```

Transfer learning example

- Define image preprocessing and obtain training and validation batches from the dataset

```
15 def format_image(image, label):  
16     image = tf.image.resize(image, (224, 224))/255.0  
17     return image, label
```

```
18  
19 num_examples = info.splits['train'].  
20  
21 train_batches = train_examples.cache  
22     num_examples//4).map(format  
23 validation_batches = validation_exam  
24     format_image).batch(32).prefetch(1)  
25
```

Define image preprocessing function that returns the image resized to 224x224 pixels

Transfer learning example

- Define image preprocessing and obtain training and validation batches from the dataset

```
15 def format_image(image, label):  
16     image = tf.image.resize(image,  
17     return image, label  
18  
19 num_examples = info.splits['train'].num_examples  
20  
21 train_batches = train_examples.cache().shuffle(  
22     num_examples//4).map(format_image).batch(32).prefetch(1)  
23 validation_batches = validation_examples.map(  
24     format_image).batch(32).prefetch(1)  
25
```

Get the training and validation batches from the dataset

Transfer learning example

- Download pre-trained ResNet50 and add the new head

```
29 base_model = tf.keras.applications.ResNet50(  
30     weights='imagenet', include_top = False)  
31  
32 x = base_model.output  
33 x = tf.keras.layers.GlobalAveragePooling2D()(x)  
34 x = tf.keras.layers.Dense(1024, activation='relu')(x)  
35 x = tf.keras.layers.Dense(512, activation='relu')(x)  
36 preds = tf.keras.layers.Dense(2, activation='softmax')(x)  
37  
38 model = tf.keras.models.Model(inputs=base_model.input, outputs=preds)  
39
```

Transfer learning example

- Download pre-trained ResNet50 and add the new head

```
29 base_model = tf.keras.applications.ResNet50(  
30     weights='imagenet', include_top = False)  
31  
32 x = base_model.output  
33 x = tf.keras.layers.GlobalAveragePooling2D(x)  
34 x = tf.keras.layers.Dense(1024, activation='relu')(x)  
35 x = tf.keras.layers.Dense(512, activation='relu')(x)  
36 preds = tf.keras.layers.Dense(2, activation='softmax')(x)  
37  
38 model = tf.keras.models.Model(inputs=base_model.input, outputs=preds)  
39
```

Load ResNet50 model pre-trained
on ImageNet dataset, exclude head

Transfer learning example

- Download pre-trained ResNet50 and add the new head

```
29 base_model = tf.keras.applications.ResNet50(  
30     weights='imagenet', include_top = False)  
31  
32 x = base_model.output  
33 x = tf.keras.layers.GlobalAveragePooling2D()(x)  
34 x = tf.keras.layers.Dense(1024, activation='relu')(x)  
35 x = tf.keras.layers.Dense(512, activation='relu')(x)  
36 preds = tf.keras.layers.Dense(2, activation='softmax')(x)  
37  
38 model = tf.keras.models.Model(inputs=base_model.inputs, outputs=preds)  
39
```

Add new head with global average pooling layer and three dense layers

Transfer learning example

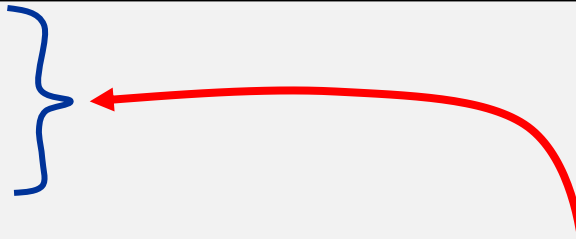
- Freeze the original model backbone layers and train the network

```
40 for layer in model.layers[:175]:
41     layer.trainable = False
42
43 for layer in model.layers[175:]:
44     layer.trainable = True
45
46 model.compile(optimizer='Adam',
47               loss='sparse_categorical_crossentropy',
48               metrics=['accuracy'])
49
50 history = model.fit(train_batches, epochs = 5,
51                     validation_data = validation_batches)
52
```

Transfer learning example

- Freeze the original model backbone layers and train the network

```
40 for layer in model.layers[:175]:
41     layer.trainable = False
42
43 for layer in model.layers[175:]:
44     layer.trainable = True
45
46 model.compile(optimizer='Adam',
47               loss='sparse_categorical_crossentropy',
48               metrics=['accuracy'])
49
50 history = model.fit(train_batches, epochs = 5,
51                     validation_data = validation_batches)
52
```



Set the first 175 layers non-trainable
and the last layers (head) trainable

Transfer learning example

- Freeze the original model backbone layers and train the network

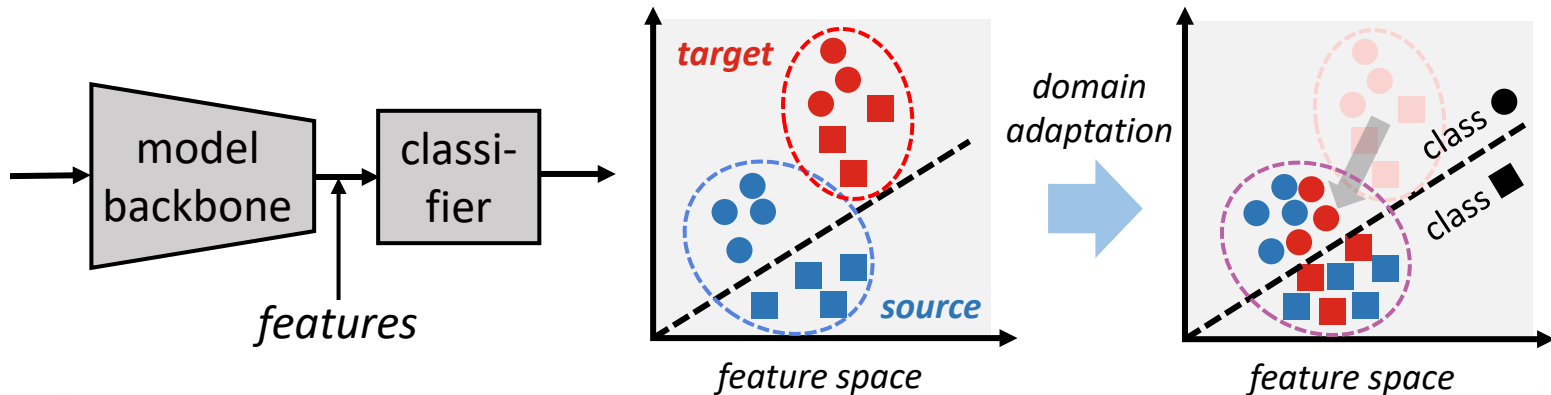
```
40 for layer in model.layers[:175]:
41     layer.trainable = False
42
43 for layer in model.layers[175:]:
44     layer.trainable = True
45
46 model.compile(optimizer='Adam',
47               loss='sparse_categorical_crossentropy',
48               metrics=['accuracy'])
49
50 history = model.fit(train_batches, epochs = 5,
51                    validation_data = validation_batches)
52
```

Train the model for 5 epochs using the training and validation data

Break: any questions?

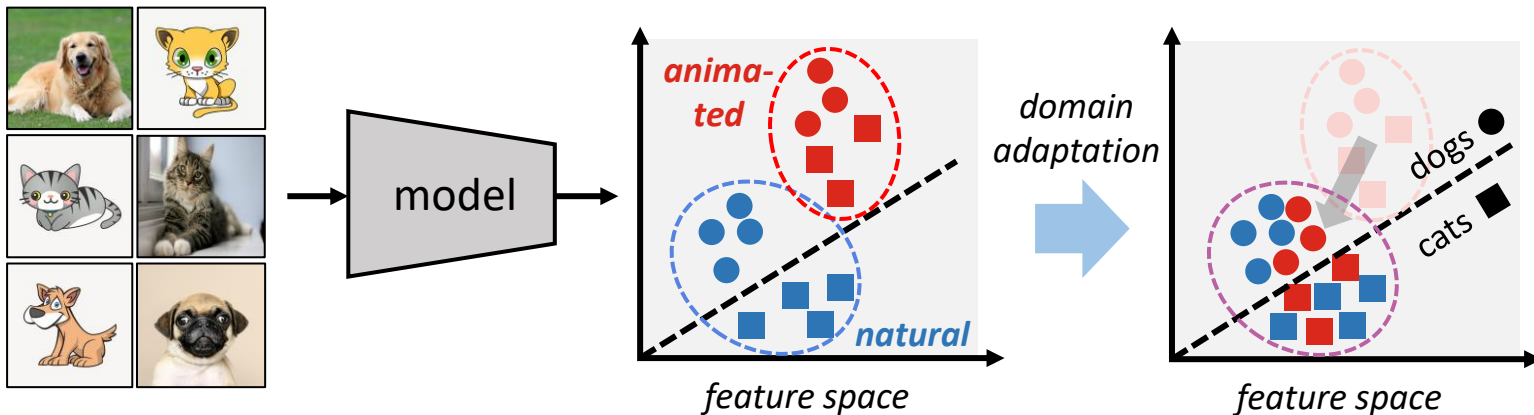
Domain adaptation

- *Domain adaptation (DA)* refers specifically to techniques adjusting the model so that the distribution of features in the target domain corresponds to the features in the source domain
 - DA can be unsupervised, weakly supervised, semi-supervised, or supervised



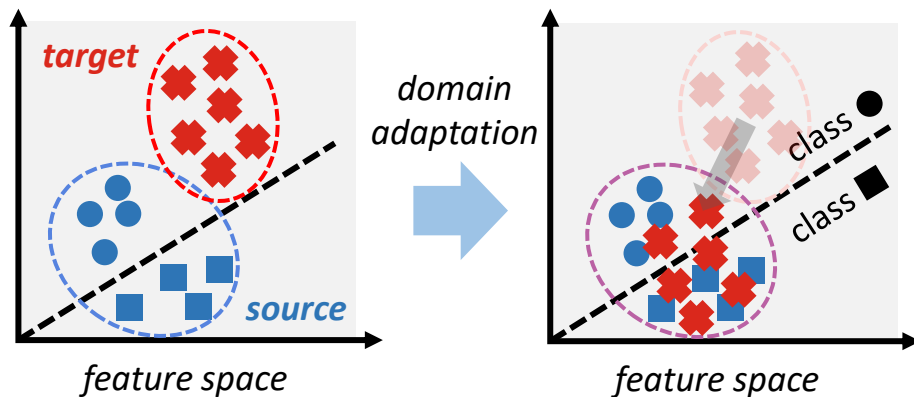
DA example scenario

- We have a model that is trained to classify pictures of cats and dogs
 - However, the model is trained on natural images, so it cannot classify animated cats and dogs correctly
 - The goal of DA is to adapt from natural image domain to animated domain



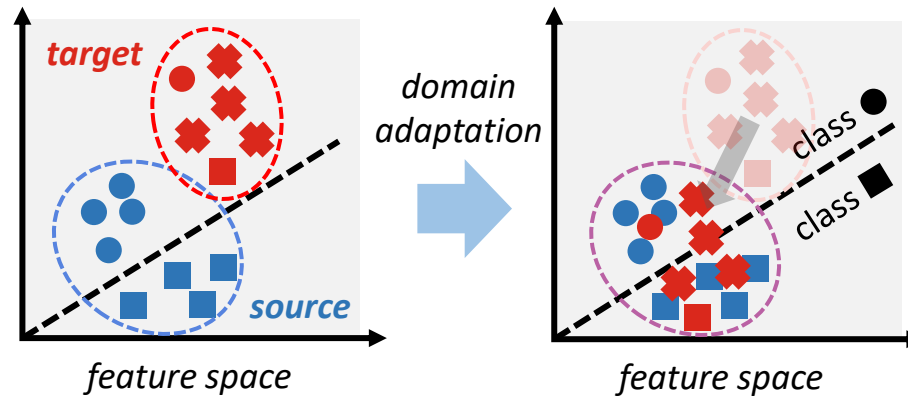
Unsupervised DA

- In unsupervised DA, only the source data is labeled
 - We have target data available, but we do not know the ground truth labels: however, we can try to make the source and target distributions similar
 - Most relevant case for DA, since transfer learning is not applicable



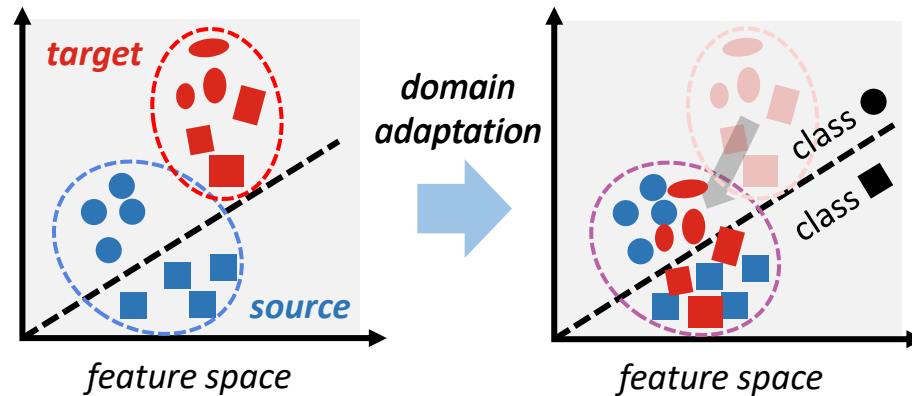
Semi-supervised DA

- Source data and some of the target data is labeled
 - A lot of target data may be available, but only part of it is labeled
 - The labeled target data can be used to improve DA in comparison with unsupervised DA



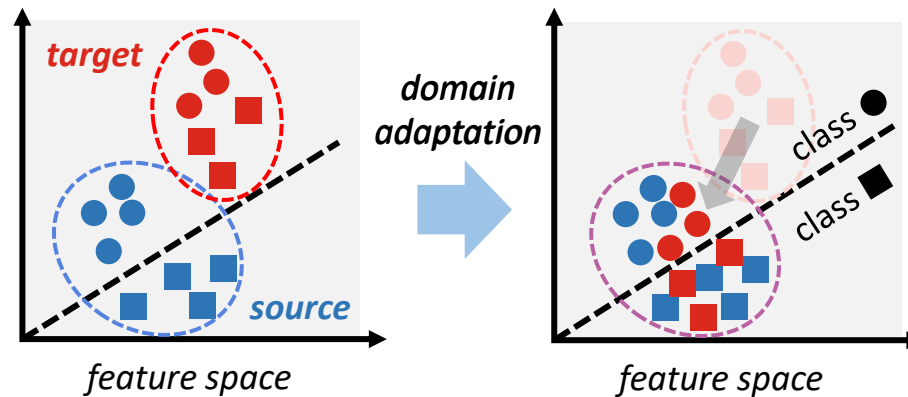
Weakly supervised DA

- In weakly supervised DA, there is some label information for the target domain data, but it can be noisy or incomplete
 - Weakly supervised DA can benefit from the partial information available



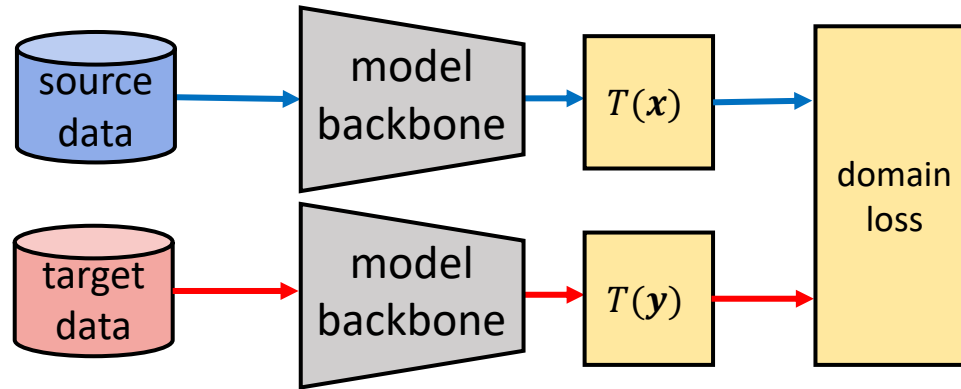
Supervised DA

- Source data and target data are both fully labeled
 - Usually, transfer learning and fine-tuning could be used
 - However, supervised DA can be useful for *domain alignment* as it accounts the feature distributions in addition to the class labels



Feature transformation

- Some kind of trainable feature transformation $T(\cdot)$ could be applied to the feature vectors to shift the distributions
 - Domain loss defined to measure differences between distributions
 - However, minimising domain loss could affect classification accuracy



Maximum mean discrepancy (MMD)

- *Maximum mean discrepancy (MMD)* often used to measure the difference between two distributions in machine learning
 - Can be used as domain loss function in DA
 - Compares the means of two sample vectors $\mathbf{x} = \{x_1, \dots, x_n\}$ and $\mathbf{y} = \{y_1, \dots, y_m\}$ after using mapping $\varphi(\cdot)$ to a particular Hilbert space

$$MMD(\mathbf{x}, \mathbf{y}) = \left\| \frac{1}{n} \sum_{i=1}^n \varphi(x_i) - \frac{1}{m} \sum_{i=1}^m \varphi(y_i) \right\| =$$

$$\left(\sum_{i,j=1}^n \frac{k(x_i, x_j)}{n^2} + \sum_{i,j=1}^m \frac{k(x_i, y_j)}{n^2} - 2 \sum_{i,j=1}^{n,m} \frac{k(x_i, x_j)}{nm} \right)^{\frac{1}{2}}$$

Squared MMD

- Many practical applications use squared MMD approximated as:

$$MMD^2(\mathbf{x}, \mathbf{y}) = \frac{1}{n^2} \sum_{j=1}^n \sum_{i=1}^n \frac{k(x_i, x_j)}{n^2} + \sum_{i,j=1}^m \frac{k(y_i, y_j)}{m^2} - 2 \sum_{i,j=1}^{n,m} \frac{k(x_i, y_j)}{nm}$$

- Given Gaussian kernel $k(x_i, x_j) = \exp\left(\frac{-\|x_i - x_j\|^2}{2\sigma^2}\right)$

$$MMD^2(\mathbf{x}, \mathbf{y}) =$$

$$\frac{1}{n^2} \sum_{i,j=1}^n \exp\left(\frac{-\|x_i - x_j\|^2}{2\sigma^2}\right) + \frac{1}{m^2} \sum_{i,j=1}^m \exp\left(\frac{-\|y_i - y_j\|^2}{2\sigma^2}\right) - \frac{1}{nm} \sum_{i,j=1}^{n,m} \exp\left(\frac{-\|x_i - y_j\|^2}{2\sigma^2}\right)$$

Other distribution distances

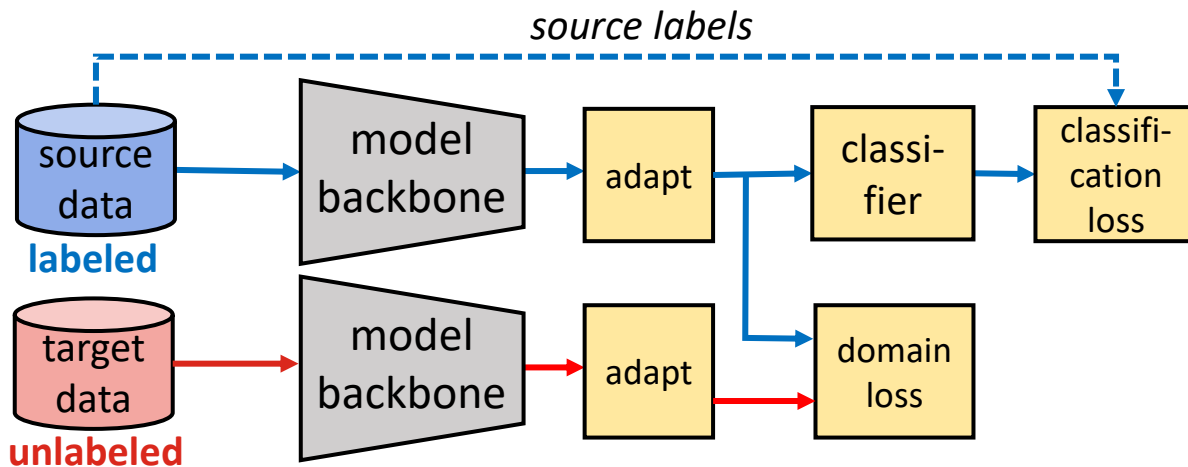
- Other distribution distance metrics could also be used to measure domain loss, given probability distributions $t(x)$ for target and $s(x)$ for the source domain, for example:

- Kullbeck-Leibler (KL) Divergence:*
$$D_{KL}(s||t) = \int s(x) \log \left(\frac{s(x)}{t(x)} \right) dx$$

- Hellinger distance:*
$$D_H^2(s||t) = \int (\sqrt{s(x)} - \sqrt{t(x)})^2 dx$$

Deep domain confusion

- Adaptation layer to make source and target domain distributions similar
 - Squared domain loss (MMD^2) subject to minimise
 - Traditional classification loss used to ensure classification accuracy



Combining domain and classification loss

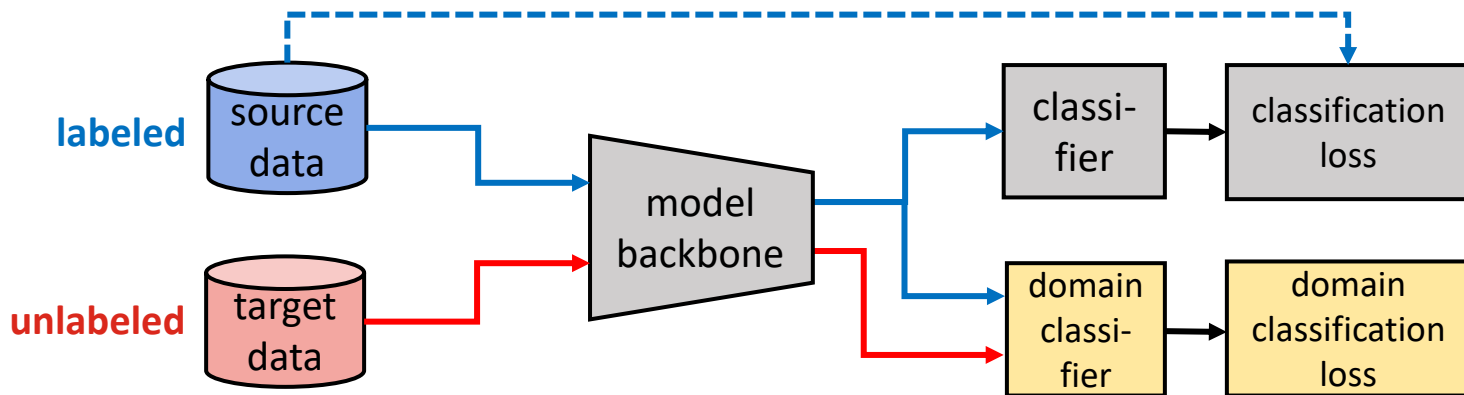
- In deep domain confusion, we can combine classification loss $\mathcal{L}_C$ and MMD loss using labeled data X_L , ground truth labels L , source data X_S , target data X_T , and hyperparameter λ into total loss $\mathcal{L}$:

$$\mathcal{L} = \mathcal{L}_C(X_L, L) + \lambda \text{MMD}^2(X_S, X_T)$$

- Hyperparameter λ adjusts the relative importance of domain loss
- In the unsupervised case, labeled data would consist of X_S , but we could also include a labeled subset of X_T in X_L for semi-supervised learning

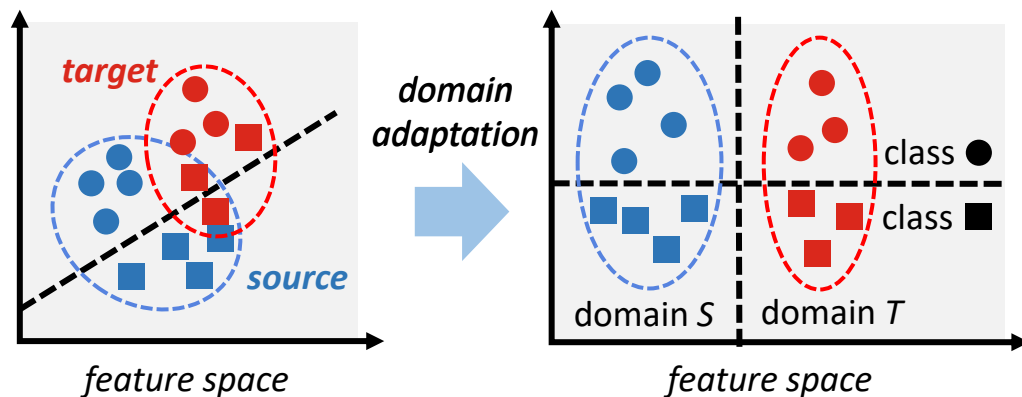
Domain adversarial networks

- In *domain adversarial networks*, the network is fine-tuned for two tasks: conventional classification, and domain classification
 - Expected to find the best trade-off between features that can discriminate different domains and performs also well in the original classification task



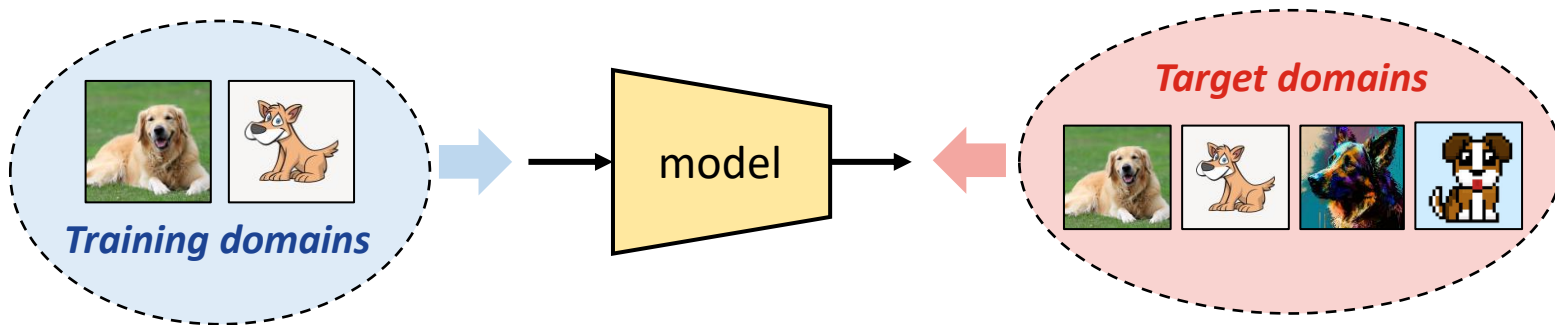
Domain adversarial network example

- The model will learn to discriminate classes *and* domains
 - Even though there are no class labels for the target domain, we can assume that training for two tasks will improve classification performance in both domains



Domain generalisation

- The domain adaptation methods explained above require labeled source domain data and target domain data labeled at least for the domain
 - However, it would be beneficial if the model could generalise to all domains even without any data available representing the target domain!
- *Domain generalisation (DG)* aims to generalise to unseen domains

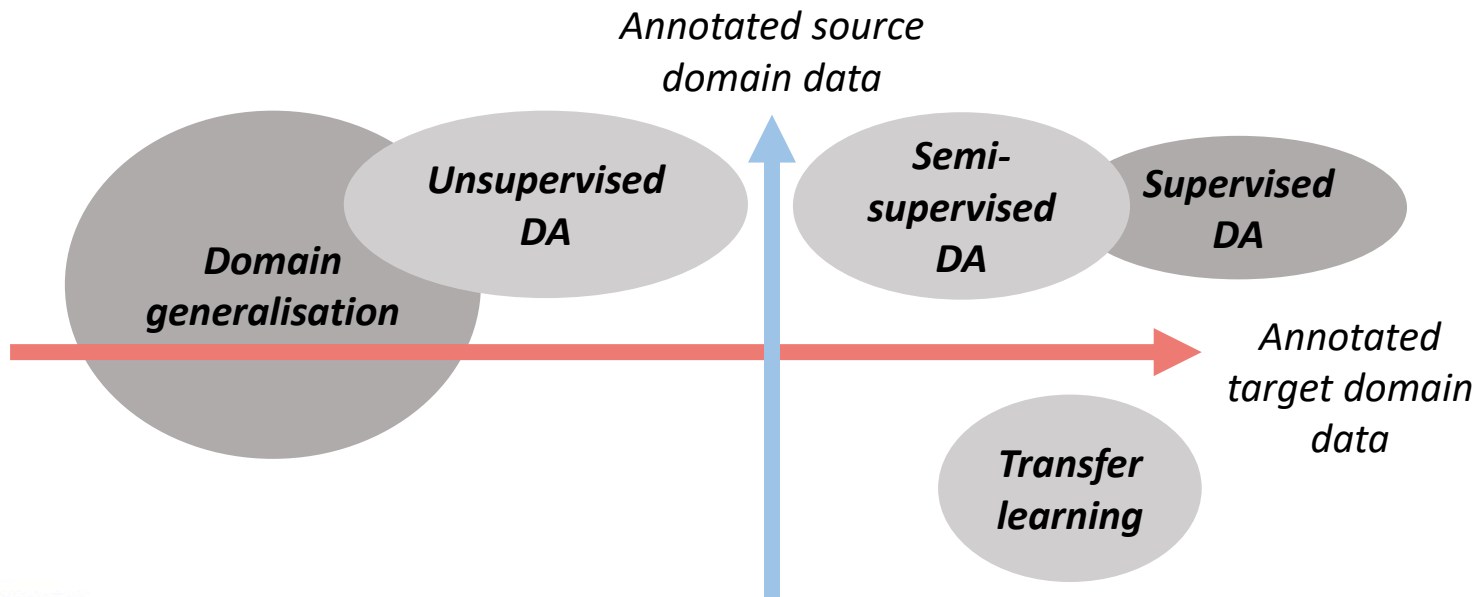


Common DG approaches

- *Domain alignment* is expected to improve model generalisation ability
 - Model trained using multiple domains; MMD or some other domain loss metric applied to align distributions for different domains
- *Data augmentation* applies transformations to the source data
 - Increases source data diversity and can mimic different unseen domains
- *Ensemble learning* trains different versions of the same model
 - Different initialisation or training data used for training the models
 - Predictions are formed by combining predictions from different models

Choice of DA strategy

- The choice of DA strategy depends mostly on the availability of data



Summary

- Domain adaptation and transfer learning are essential techniques to re-purpose deep pre-trained models
 - In many situations, training data is limited: good performance cannot be achieved by training from scratch
- Transfer learning and fine-tuning are used to re-train a model
 - Similar to normal supervised training process, but pre-trained model weights are used as a starting point
- Domain adaptation tunes the internal representations (features)
 - Can be used to shift the distributions of source and target domains even in situations where labeled target domain data is not available

Questions and discussion